

VLSI Project 1 Report

2022171005 권민찬

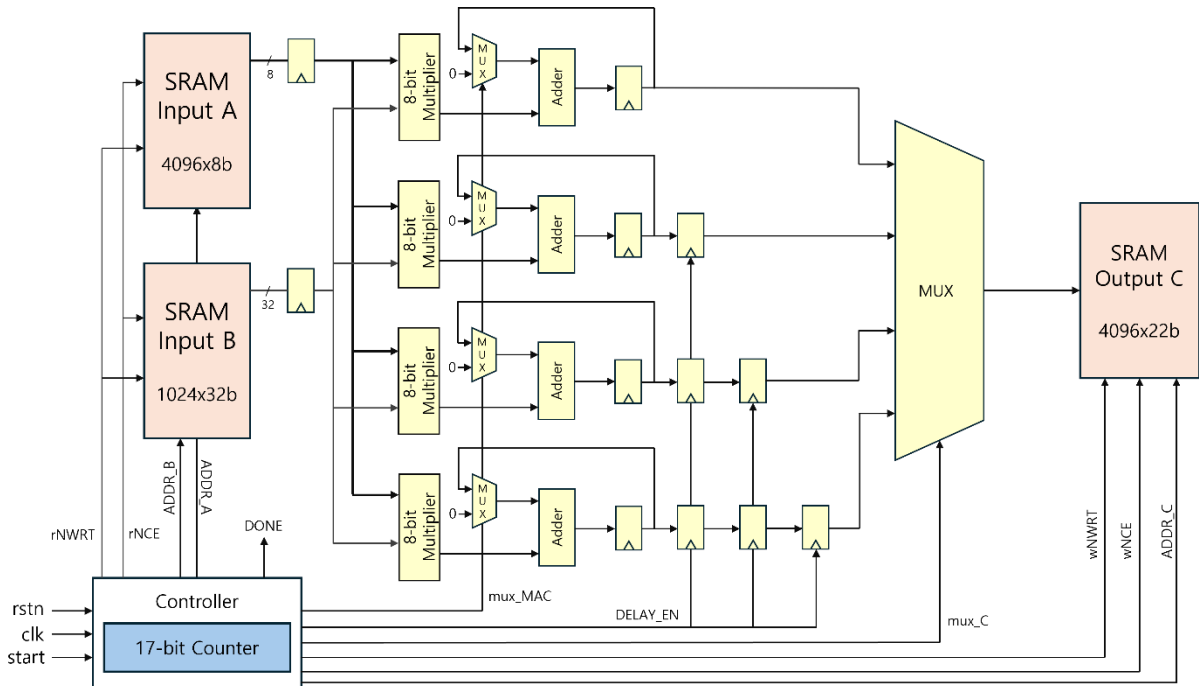
1. Controller Design

i. Matrix Multiplication

행렬곱 연산은 $O(n^3)$ 의 시간복잡도를 가지고 있다. 따라서 64×64 행렬을 곱하는 연산을 수행하기 위해서는 2^{18} 사이클이 필요하다. 이번 프로젝트에서는 4개의 MAC을 사용하여 Throughput을 4배 증가시켰기 때문에, 연산을 완료하기 위해 2^{16} 사이클이 필요하다. 따라서 행렬곱 연산만을 위해서는 최소 16비트 카운터 1개가 필요하다는 것을 알 수 있다.

이 프로젝트에서는 두 개의 메모리에 저장된 데이터를 읽어오고, 이를 이용해 MAC 연산을 수행한 후, 다시 출력 메모리에 쓰기를 하는 과정까지 구현해야 한다. 따라서 컨트롤러는 각 메모리에 적절한 주소와 제어 신호를 인가하고, 레지스터와 MUX에도 적절한 제어신호를 인가해야 한다.

ii. Architecture



탑 모듈의 회로도도 위의 그림과 같다. 컨트롤러는 카운터를 기반으로 세 메모리의 주소 (ADDR_A, ADDR_B, ADDR_C)와 메모리의 읽기 신호 (rNWRT, wNWRT), enable 신호

(rNCE, wNCE)를 인가해줘야 한다. 또한 곱셈 결과를 누적할지, 0으로 초기화할지 결정하는 MUX와 메모리에 어떤 데이터를 쓸지 결정하는 MUX에 선택 신호 (mux_MAC, mux_C)를 인가해줘야 한다. 연산과 쓰기가 모두 마무리된 후 DONE 신호까지 제어해야 한다.

메모리 A, B에서 읽은 데이터를 안정적으로 이용하기 위해 메모리 읽기 포트와 각 MAC 유닛 사이에 레지스터를 추가하였다. 또한, 출력을 저장하는 메모리가 싱글포트이기 때문에, 한 번에 MAC 하나의 결과만을 쓸 수 있어 4개의 MAC 결과가 차례대로 메모리의 쓰기포트에 전달될 수 있도록 MAC의 끝에 레지스터를 순차적으로 달아주었다. MAC 결과를 지연시키기 위해 도입한 레지스터에 쓰레기 값이 쓰여지는 것을 방지하기 위해 컨트롤러 DELAY_EN이라는 레지스터 enable 신호를 출력하도록 했다.

iii. Counter Bit Width

이론적으로 행렬곱 연산만을 수행하기 위해서는 16비트 카운터만 있으면 된다. 하지만 쓰기 동작은 행렬곱 연산을 통해 4개의 원소가 만들어진 다음에 진행될 수 있기 때문에, 마지막 4개의 원소를 쓰기 위해서는 $2^{16}+4$ 사이클이 필요하게 된다. 16비트 카운터를 쓰면 $2^{16} - 1$ 이후에 카운터 값이 다시 0으로 wrap around되기 때문에 마지막 ADDR_C를 계산하기도 어려워지고, 카운터 값만을 보고 행렬곱 연산이 끝난 것인지, 시작하는 것인지 구분하기 어려워 DONE 신호를 인가하기도 어려워진다.

이를 해결하기 위해 주소 카운터를 12비트로 줄이고 별도의 FSM 상태 카운터를 도입해 보는 것도 고려했으나, 17비트 글로벌 카운터를 1개 사용하는 것보다 더 많은 레지스터를 필요로 했기 때문에 최종적으로는 17비트 카운터를 이용하여 설계했다.

iv. Address Logic

- Address A

A는 row-wise 메모리 접근 패턴을 보인다. 즉, 0부터 시작해 주소를 1씩 늘려가며 63까지 증가했다가, 다시 0으로 초기화되어 같은 규칙으로 메모리에 접근을 하게 된다. 이것이 16번 반복되어야 한다. 그 후에는 주소 64부터 동일한 접근 패턴을 보인다. 이런 접근 패턴을 구현하기 위해서 ADDR_A의 하위 6비트는 카운터의 하위 6비트와 같고, 상위 6비트는 counter[15:10]와 같아야 한다. ADDR_A = {counter[15:10], counter[5:0]}으로 assign을 하면 행렬 A의 원소에 접근할 수 있다.

- Address B

B는 column-wise 메모리 접근 패턴을 보인다. 메모리에는 원소들이 row-wise로 저장되어 있기 때문에 0, 16, 32, 48, 64, ...의 접근패턴을 보이며, 주소가 16만큼 차이나는 접근을 64번 반복한 후 1, 17, 33, 49, 65, ...으로 같은 접근 패턴을 16번 반복하게 된다. 따라서 하위 4비트를 offset으로 취급하여 64사이클마다 1씩 증가시키고, 상위 6비트는 매 사이클마다 1씩 증가시키면 되기 때문에 ADDR_B = {counter[5:0], counter[9:6]}로 assign 하면 행렬 B의 원소에 적절하게 접근할 수 있다.

- Address C

출력되는 결과를 저장하기 위해 주소를 만드는 것이 제일 어려운 부분이었다. 64사이클마다 유의미한 결과가 출력되기 때문에 64사이클마다 ADDR_C를 1씩, 4번 증가시켜야 한다. 즉, 64사이클마다 ADDR_C의 하위 2비트가 00, 01, 10, 11을 반복하게 된다. Start와 RUN 신호가 인가된 후로부터 MAC의 누적 레지스터에 곱셈의 결과가 latch되기 위해서는 3사이클이 걸리게 된다. 메모리 읽기가 synchronous하고, MAC의 입력단에 레지스터가 있고, 누적을 위해 출력단에 레지스터가 있기 때문이다. 카운터가 0부터 2일 때까지는 누적 레지스터에 아무런 값이 쓰이지 않고, 카운터가 3부터 MAC 연산을 반복하여 66이 되면 연산이 완료되어 MAC의 출력에 값이 latch되고, 카운터가 67일 때 최초로 메모리에 연산값이 쓰이게 된다. 이것이 가능하게 하기 위해서는 카운터가 66일 때 C 메모리 주소가 출력되어야 한다.

이런 메모리 접근 패턴을 구현하려면 카운터에서 66을 빼고, 그 중 일부 비트만 와이어로 연결해주면 된다. 하지만 이럴 경우 17비트 카운터 값에서 66을 빼기 위해 거대한 감산기가 합성될 것이고, CPD와 면적을 늘리는 원인이 될 것이다.

C 주소의 하위 2비트는 0, 1, 2, 3이 계속 반복되는 구조이기 때문에 카운터의 하위 2비트를 이용한 단순 조합논리로 만들 수 있다. 이렇게 하면 17비트 카운터 값에서 1'd2을 빼는 것은 해결됐기 때문에 64를 빼주기만 하면 된다. 이를 아래처럼 나타낼 수 있다. 결국 count에서 64를 빼는 연산은 count에서 1을 빼고, shift를 하는 연산과 동일하다. 이를 이용하면 매우 효율적으로 C 주소 생성 논리를 구현할 수 있다.

$$\text{count} - 64 = \text{count}[16:6] * 64 + \text{count}[5:0] - 64 = (\text{count}[16:6] - 1) * 64 + \text{count}[5:0]$$

```
wire [9:0] c_upper = counter[15:6] - 10'b1;
wire      c_offset_h = ~counter[1];
wire      c_offset_l = counter[0];
assign ADDR_C = {c_upper, c_offset_h, c_offset_l};
```

Counter[15:6] - 10'b1 부분도 감산기가 합성된다. Counter[6]=0일 때만 감산기가 필요하기 때문에, 이를 이용해 최적화를 해볼 수도 있었지만, 최적화를 하더라도 상위 비트에서

연쇄적으로 1을 빌려와야 하는 논리를 구현해야 하기 때문에 그대로 감산기로 구현하는 것과 큰 차이가 없어 단순하게 작성했다.

v. Control Signals

Controller의 주된 역할은 MAC과 메모리에 적절한 제어 신호를 인가하는 것이다. 컨트롤러가 인가해야 할 주요 제어 신호와 그 신호를 출력하기 위한 논리는 다음과 같다.

- RUN, Counter

Start가 한 사이클 동안만 켜지고 꺼지기 때문에 행렬 곱셈을 진행한다는 상태를 나타내기 위해 RUN 레지스터를 추가했다. RUN은 Start가 켜지면 켜지고, 최종 쓰기까지 완료하면 꺼진다. 또한 17비트 카운터 값은 0으로 초기화되어 RUN이 켜지고 한 사이클 뒤부터 1씩 증가하게 된다.

- wNCE, wNWRT

66번째 사이클에서 첫번째 결과가 MAC의 출력에 맷히게 된다. 67번째 사이클에서 메모리에 올바르게 쓰기가 진행되려면 66번째 사이클에 C 메모리를 enable하고, 쓰기 모드로 설정해야 한다. 따라서 카운터가 65부터 68일 때까지 순차 논리로 wNCE, wNWRT를 모두 0으로 설정하여 66번째 사이클부터 4사이클동안 올바른 메모리 제어 신호가 출력될 수 있도록 했다.

이를 구현하기 위해 counter[5:0]을 1, 2, 3, 4와 대조했는데, 카운터 전체 비트가 1, 2, 3, 4일 때는 쓰기를 하면 안 되기 때문에 예외처리를 위한 논리도 포함했다.

- DELAY_EN

4개의 MAC의 출력에는 동시에 연산 결과가 맷히지만, 메모리에 쓰는 것은 순차적으로 진행하기 때문에 이를 위해 MAC의 출력을 1, 2, 3사이클 지연시키는 레지스터를 추가했다. 이 레지스터들에는 enable (DELAY_EN) 신호가 있어서 완성되지 않은 연산 중간값이 쓰여지는 것을 방지하고 있다. DELAY_EN은 wNCE, wNWRT와 동일한 논리를 이용하면 적절한 값을 인가할 수 있다.

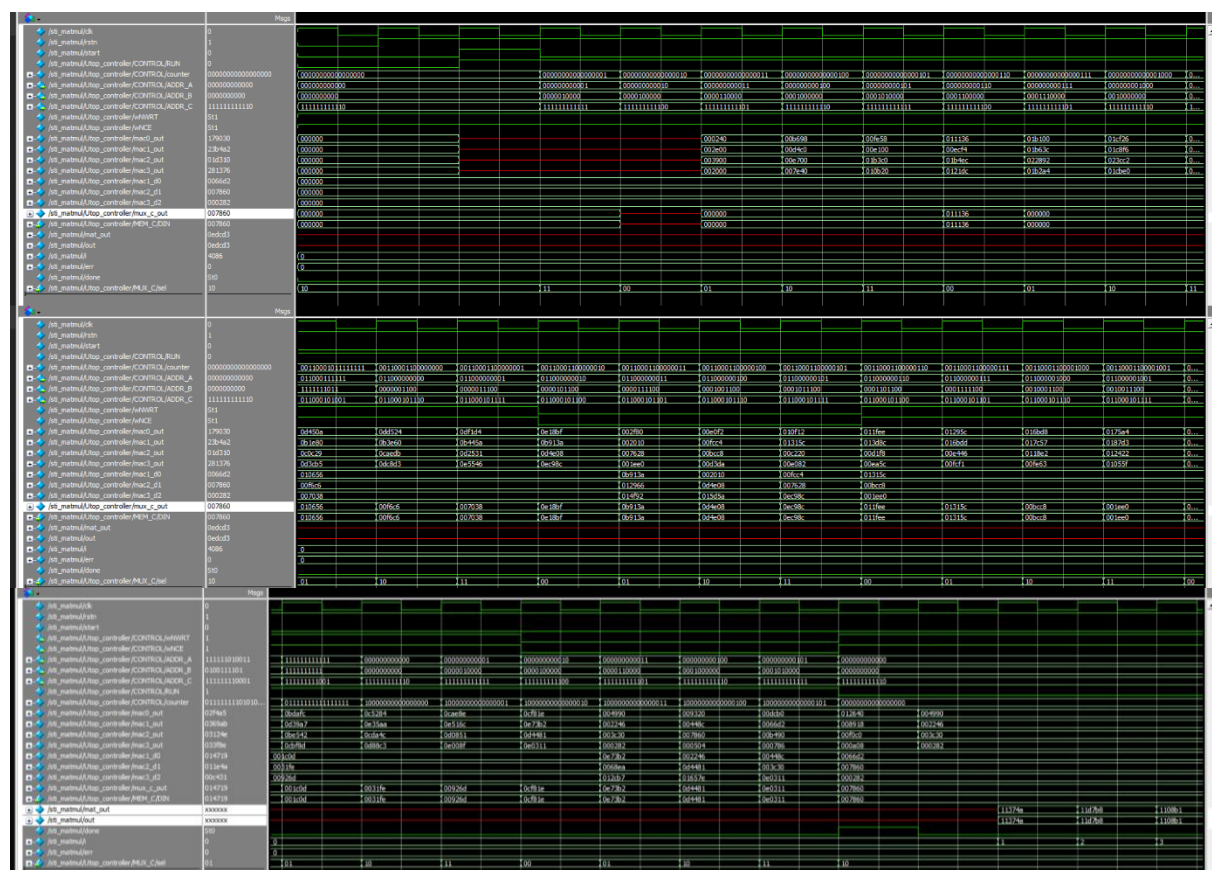
- MUX_MAC, MUX_C

MAC 내부에 있는 MUX와 4개의 MAC 결과를 선택하여 메모리에 인가하는 MUX를 위한 제어 신호도 필요하다.

MAC 내부의 MUX 제어 신호 MUX_MAC는 메모리 쓰기가 시작될 때 (counter[5:0]==6'b1) 0을 인가하여 기존 연산값을 초기화하고, 그 외의 경우에는 1을 인가하여 누적이 계속될 수 있게 하였다.

- DONE

2. Functional Verification



위 그림은 행렬곱 연산을 수행하는 과정과 결과를 나타낸 파형이다. Start 신호는 한 사이클 동안만 인가되고 다시 꺼지기 때문에 RUN이라는 별도의 신호가 행렬 곱셈을 하는 동안 계속 1을 유지하고 있다. RUN 신호가 켜진 뒤 3사이클이 지나서 partial product가 MAC의 레지스터에서 누적되기 시작하는 것을 첫번째 파형 그림에서 볼 수 있다.

가운데 파형에서 wNWRT, wNCE가 0으로 인가되면서 delay line에 있는 레지스터의 값이 차례대로 변하고, mux_c_out에서 적절한 결과가 출력되면서 메모리에 올바른 값이 쓰이는 것을 확인할 수 있다.

마지막으로 모든 연산이 종료되면 DONE이 1로 바뀌었다가 다시 토글되어 꺼지는 것을 볼 수 있다. DONE이 1로 출력된 이후에는 더 이상 행렬곱 연산이나 카운터 증가가 일어나지 않아, 전력이 누설되는 것을 방지할 수 있다.

위 그림에서 볼 수 있듯이 C 메모리의 마지막 데이터까지 비교했을 때 정답과 같았으며, 테스트벤치 종료 시점에 err가 0이다.

3. Synthesis Results

i. PPA Results

- Timing

Startpoint: A_IN_REG/q_reg[2] (rising edge-triggered flip-flop clocked by clk) Endpoint: M0/acc_reg[17] (rising edge-triggered flip-flop clocked by clk) Path Group: clk Path Type: max			Startpoint: A_IN_REG/q_reg[5] (rising edge-triggered flip-flop clocked by clk) Endpoint: M2/acc_reg[20] (rising edge-triggered flip-flop clocked by clk) Path Group: clk Path Type: max		
Point	Incr	Path	Point	Incr	Path
clock clk (rise edge)	0.00	0.00	clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00	clock network delay (ideal)	0.00	0.00
A_IN_REG/q_reg[2]/CLK (dffcs2)	0.00	0.00 r	A_IN_REG/q_reg[5]/CLK (dffcs2)	0.00	0.00 r
A_IN_REG/q_reg[2]/QN (dffcs2)	0.23	0.23 r	A_IN_REG/q_reg[5]/QN (dffcs2)	0.23	0.23 r
A_IN_REG/q_reg[2]/Q (dffcs2)	0.18	0.40 f	A_IN_REG/q_reg[5]/Q (dffcs2)	0.12	0.34 f
M0/A[2] (MAC_0)	0.00	0.40 f	M2/A[5] (MAC_2)	0.00	0.34 f
M0/mult_13/a[2] (MAC_0_DW_mult_uns_0)	0.00	0.40 f	M2/mult_13/a[5] (MAC_2_DW_mult_uns_0)	0.00	0.34 f
M0/mult_13/U252/Q (l1s3)	0.21	0.61 r	M2/mult_13/U250/Q (l1s1)	0.26	0.60 r
M0/mult_13/U268/Q (nor2s2)	0.15	0.76 f	M2/mult_13/U466/Q (nor2s1)	0.28	0.88 f
M0/mult_13/U314/Q (and2s2)	0.28	1.04 f	M2/mult_13/U133/OUTS (faddis3)	0.82	1.70 r
M0/mult_13/U159/OUTC (faddis3)	0.37	1.41 f	M2/mult_13/U420/Q (xor2s1)	0.42	2.12 r
M0/mult_13/U156/OUTS (faddis3)	0.76	2.17 r	M2/mult_13/U426/Q (xor2s2)	0.42	2.55 r
M0/mult_13/U430/Q (xor2s2)	0.24	2.41 r	M2/mult_13/U359/Q (xor3s3)	0.38	2.93 r
M0/mult_13/U298/Q (aoi2is3)	0.21	2.62 f	M2/mult_13/U347/Q (nor2s2)	0.13	3.06 f
M0/mult_13/U486/Q (aoi2is3)	0.23	2.86 r	M2/mult_13/U419/Q (aoi2is3)	0.21	3.28 r
M0/mult_13/U320/Q (aoi2is3)	0.28	3.14 f	M2/mult_13/U519/Q (l1s3)	0.10	3.37 f
M0/mult_13/U361/Q (aoi2is3)	0.26	3.39 r	M2/mult_13/U377/Q (nnd2s3)	0.15	3.53 r
M0/mult_13/U429/Q (aoi2is3)	0.29	3.68 f	M2/mult_13/U372/Q (l1s6)	0.12	3.65 f
M0/mult_13/U481/Q (aoi2is2)	0.26	3.94 r	M2/mult_13/U371/Q (aoi2is3)	0.23	3.88 r
M0/mult_13/U444/Q (xnor2s2)	0.35	4.29 r	M2/mult_13/U3/Q (xnor2s3)	0.35	4.23 f
M0/mult_13/product[14] (MAC_0_DW_mult_uns_0)	0.00	4.29 r	M2/mult_13/product[14] (MAC_2_DW_mult_uns_0)	0.00	4.23 f
M0/add_21/B[14] (MAC_0_DW01_add_0)	0.00	4.29 r	M2/add_21/B[14] (MAC_2_DW01_add_0)	0.00	4.23 f
M0/add_21/U275/Q (nnd2s1)	0.17	4.46 f	M2/add_21/U63/Q (nor2s3)	0.15	4.38 r
M0/add_21/U217/Q (aoi2is3)	0.24	4.69 r	M2/add_21/U203/Q (nor2s3)	0.18	4.56 f
M0/add_21/U191/Q (aoi2is3)	0.21	4.90 f	M2/add_21/U222/Q (and2s2)	0.27	4.83 f
M0/add_21/U178/Q (nnd2s2)	0.23	5.13 r	M2/add_21/U215/Q (nnd2s3)	0.12	4.96 r
M0/add_21/U184/Q (aoi2is2)	0.26	5.39 f	M2/add_21/U198/Q (nnd2s3)	0.17	5.12 f
M0/add_21/U171/Q (xor2s2)	0.36	5.75 r	M2/add_21/U214/Q (nnd2s3)	0.12	5.25 r
M0/add_21/SUM[17] (MAC_0_DW01_add_0)	0.00	5.75 r	M2/add_21/U213/Q (and2s3)	0.23	5.47 r
M0/acc_reg[17]/CLRB (dffcs2)	0.00	5.75 r	M2/add_21/U212/Q (xnor2s2)	0.25	5.72 r
data arrival time	5.75	5.75	M2/add_21/SUM[20] (MAC_2_DW01_add_0)	0.00	5.72 r
			M2/acc_reg[20]/CLRB (dffcs1)	0.00	5.72 r
clock clk (rise edge)	6.10	6.10	data arrival time	5.72	5.72
clock network delay (ideal)	0.00	6.10	clock clk (rise edge)	6.20	6.20
M0/acc_reg[17]/CLK (dffcs2)	0.00	6.10 r	clock network delay (ideal)	0.00	6.20
library setup time	-0.48	5.62	M2/acc_reg[20]/CLK (dffcs1)	0.00	6.20 r
data required time	5.62	5.62	library setup time	-0.48	5.72
data required time	5.62	5.62	data required time	5.72	5.72
data arrival time	-5.75	-5.75	data arrival time	-5.72	-5.72
slack (VIOLATED)	-0.13	-0.13	slack (MET)	0.00	0.00

클럭 주기를 6.1ns로 설정하고 합성을 했을 때는 negative slack이 되면서 주어진 타이밍

조건을 맞추지 못했다. 다시 6.2ns로 설정하고 합성했을 때는 타이밍 조건을 충족하는 것을 확인할 수 있다. 이 회로는 161.29MHz로 동작하는 것을 알 수 있다.

- Area

<pre>***** Report : area Design : top_controller_synth Version: Z-2007.03-SP4 Date : Wed May 6 05:07:10 2026 ***** Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db) Number of ports: 4 Number of nets: 333 Number of cells: 17 Number of references: 17 Combinational area: 137255.764290 Noncombinational area: 57330.950653 Net Interconnect area: undefined (No wire load specified) Total cell area: 194586.718750 Total area: undefined</pre>	<pre>***** Report : area Design : top_controller_synth Version: Z-2007.03-SP4 Date : Wed May 6 05:06:11 2026 ***** Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db) Number of ports: 4 Number of nets: 331 Number of cells: 15 Number of references: 15 Combinational area: 138168.147026 Noncombinational area: 57015.759567 Net Interconnect area: undefined (No wire load specified) Total cell area: 195183.906250 Total area: undefined</pre>
--	--

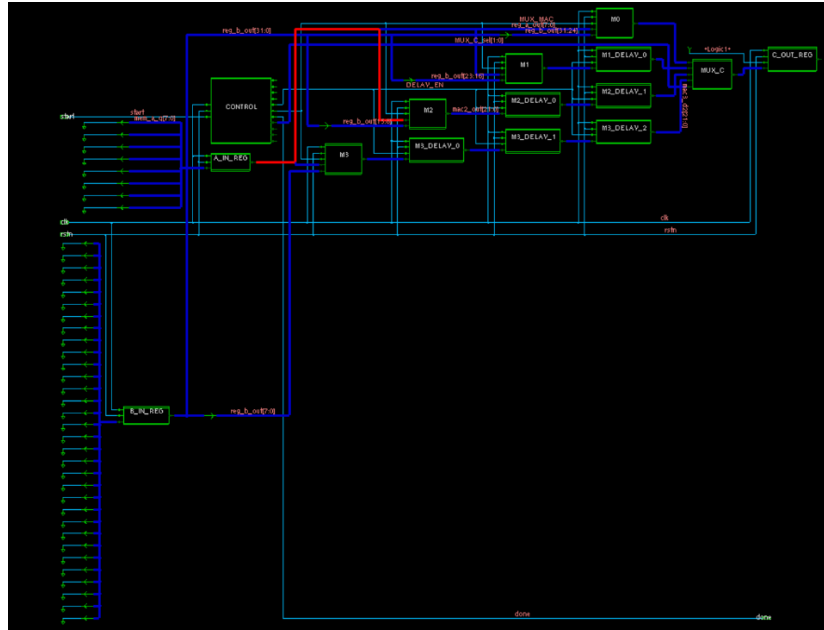
위의 두 그림은 각각 6.1ns 클럭, 6.2ns 클럭에서의 면적이다. 이번 합성에서는 평소와 다르게 클럭 주기를 크게 설정했을 때 오히려 면적이 크게 나왔다. 또한 자세히 보면 Number of nets, Number of cells, Number of references가 다른 것도 볼 수 있다. 합성을 수차례 반복해보아도 같은 결과가 나왔다. 이런 차이의 원인에 대해서는 스키매틱 항목과 discussion 항목에서 좀 더 자세히 다룰 것이다.

- Power

<pre>***** Report : power -analysis_effort low Design : top_controller_synth Version: Z-2007.03-SP4 Date : Wed May 6 05:07:11 2026 ***** Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db) Operating Conditions: nom_pvt Library: lec25dsc25_SS Wire Load Model Mode: top Global Operating Voltage = 2.25 Power-specific unit information : Voltage Units = 1V Capacitance Units = 1.000000pf Time Units = 1ns Dynamic Power Units = 1mW (derived from V,C,T units) Leakage Power Units = 1pW Cell Internal Power = 5.7143 mW (93%) Net Switching Power = 407.6776 uW (7%) ----- Total Dynamic Power = 6.1220 mW (100%) Cell Leakage Power = 1.9723 mW</pre>	<pre>***** Report : power -analysis_effort low Design : top_controller_synth Version: Z-2007.03-SP4 Date : Wed May 6 05:06:12 2026 ***** Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db) Operating Conditions: nom_pvt Library: lec25dsc25_SS Wire Load Model Mode: top Global Operating Voltage = 2.25 Power-specific unit information : Voltage Units = 1V Capacitance Units = 1.000000pf Time Units = 1ns Dynamic Power Units = 1mW (derived from V,C,T units) Leakage Power Units = 1pW Cell Internal Power = 5.5926 mW (94%) Net Switching Power = 387.1067 uW (6%) ----- Total Dynamic Power = 5.9797 mW (100%) Cell Leakage Power = 991.6154 uW</pre>
--	--

파워는 예상했던 결과가 나타났다. 클럭 주기를 6.1ns로 설정했을 때 6.1220mW, 6.2ns로 설정했을 때 5.9797mW의 total dynamic power가 나온 것을 볼 수 있다. 이는 클럭 주기를 높임으로써 전체 회로가 더 빠른 주파수에서 작동하기 때문이라고 볼 수 있다.

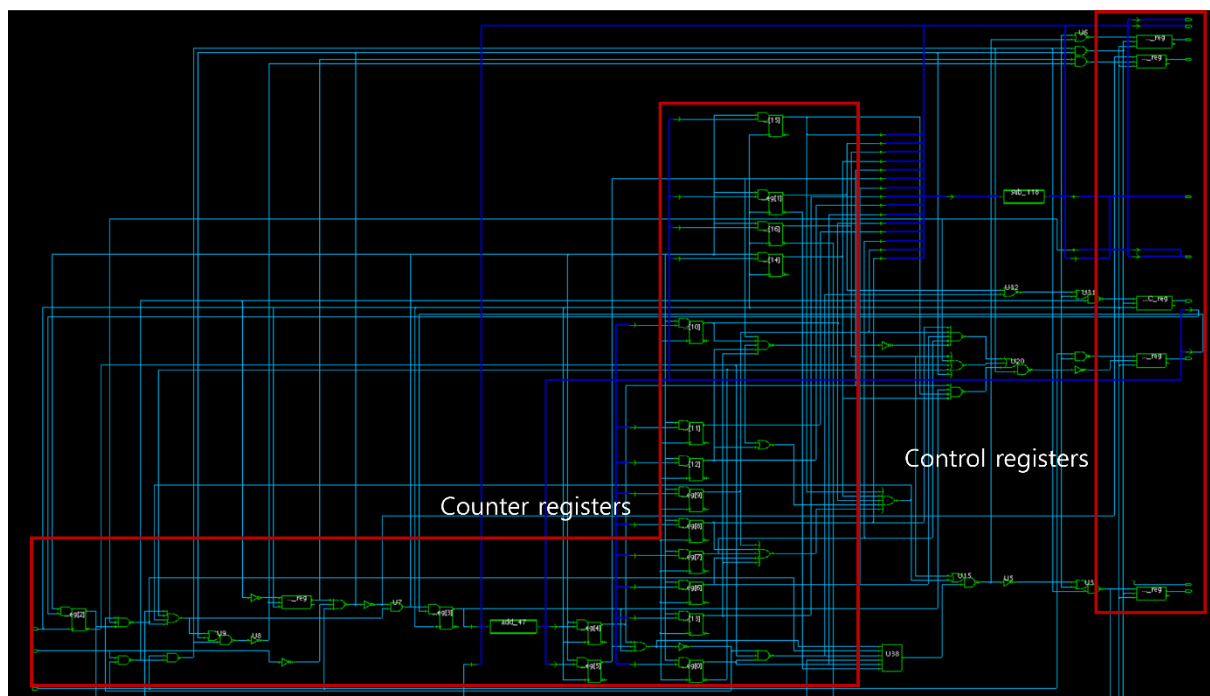
ii. Schematics



위 그림은 메모리를 제외한 모든 로직이 들어있는 탑모듈의 합성 결과이다. 탑모듈의 합성 스키매틱은 6.1ns와 6.2ns일 때 큰 차이가 없었다. 크리티컬 패스를 표시했을 때 예상했던 것과 같이 입력 레지스터인 A_IN_REG부터 MAC 유닛인 M2로 이어지는 것을 볼 수 있다. 모든 MAC 유닛 내에는 곱셈기와 덧셈기 뒤에 연산값 누적을 위한 레지스터가 있기 때문에 A_IN_REG부터 MAC 유닛 내의 레지스터까지가 크리티컬 패스라는 사실을 스키매틱에서 추론해볼 수 있다. 또한 가장 우측에 합성을 위해 추가한 C_OUT_REG를 볼 수 있다.

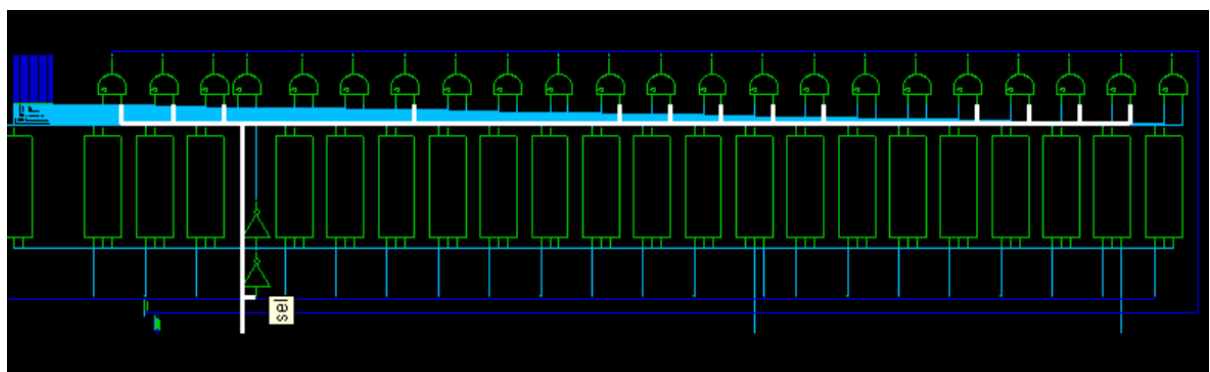
6.1ns 주기로 합성을 했을 때는 위와 같은 컨트롤러 스키매틱을 볼 수 있었다. 카운터

레지스터와 제어 신호 레지스터들이 분리되어 배치되어 있었다.



[Controller Schematic @ 6.1ns Period]

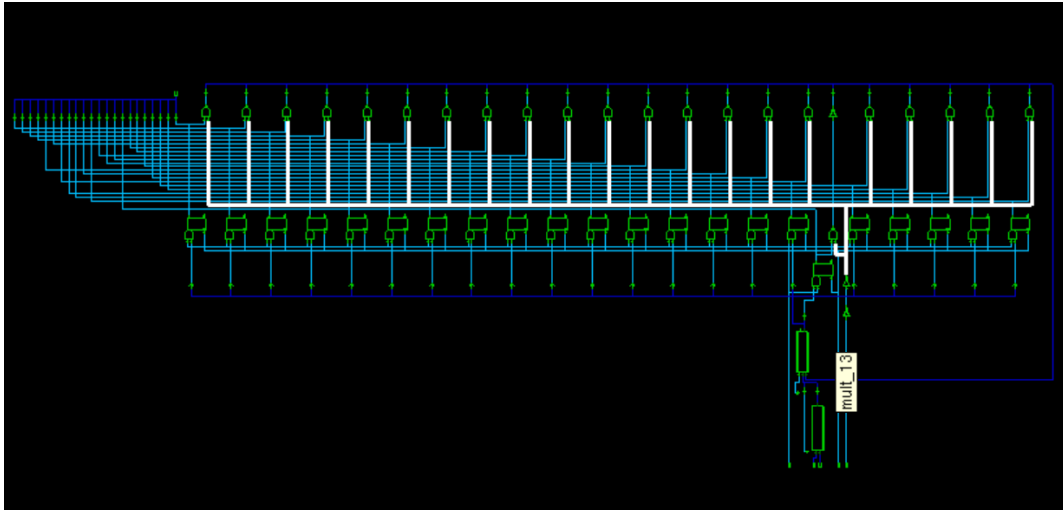
6.1ns 주기로 합성을 했을 때는 6.2ns와는 다른 스키매틱이 등장했다. 카운터 레지스터와 제어 신호 레지스터가 분리되어 있는 것은 동일했지만 이들 사이의 와이어 연결이 다른 형태를 띄고 있으며 레지스터와 조합논리 소자들이 6.2ns와는 다른 cell을 이용해서 합성된 것을 볼 수 있다.



[MAC Schematic @ 6.2ns Period, Rotated 90 Degrees]

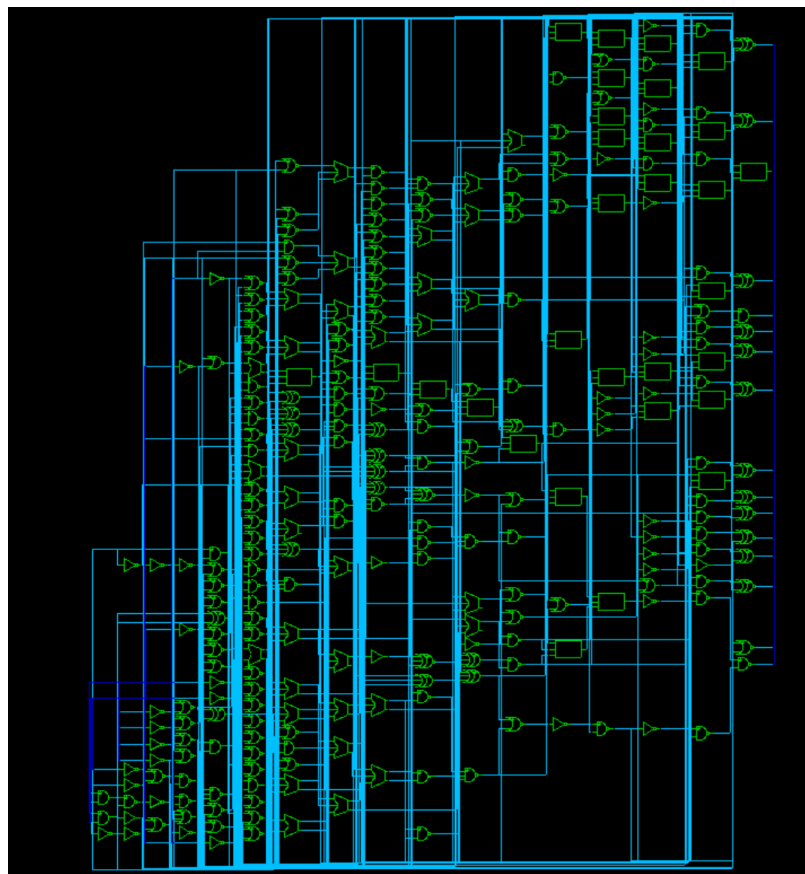
이번에는 MAC 내부이다. 6.2ns로 합성했을 때는 mux의 select 신호가 둘로 나뉘어 일부는 버퍼를 거치지 않고 전달되고 나머지 일부는 버퍼를 거쳐 전달된다. MUX의 비트폭이 넓기 때문에 회로도와는 다르게 실제 mux로 합성되지 않는 것을 확인할 수 있다.

한편 select 신호 왼쪽에 작게 표시된 두 블록이 multiplier와 accumulator이다.



[MAC Schematic @ 6.1ns Period, Rotated 90 Degrees]

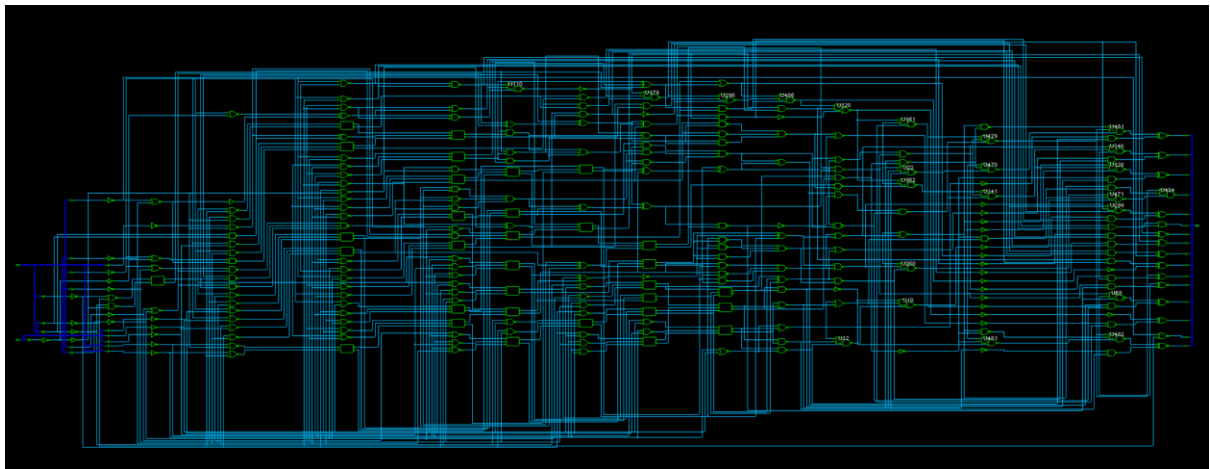
6.1ns로 합성했을 때는 또 다른 결과가 나왔다. 이번에도 사용한 cell들이 다른 것을 볼 수 있으며 흰색으로 표시된 select 신호가 전부 버퍼를 통과하여 전달되는 것을 볼 수 있다. 또한 multiplier, accumulator 모두 이번 스키매틱과 다른 크기로 보인다.



[Multiplier Schematic @ 6.2ns Period]

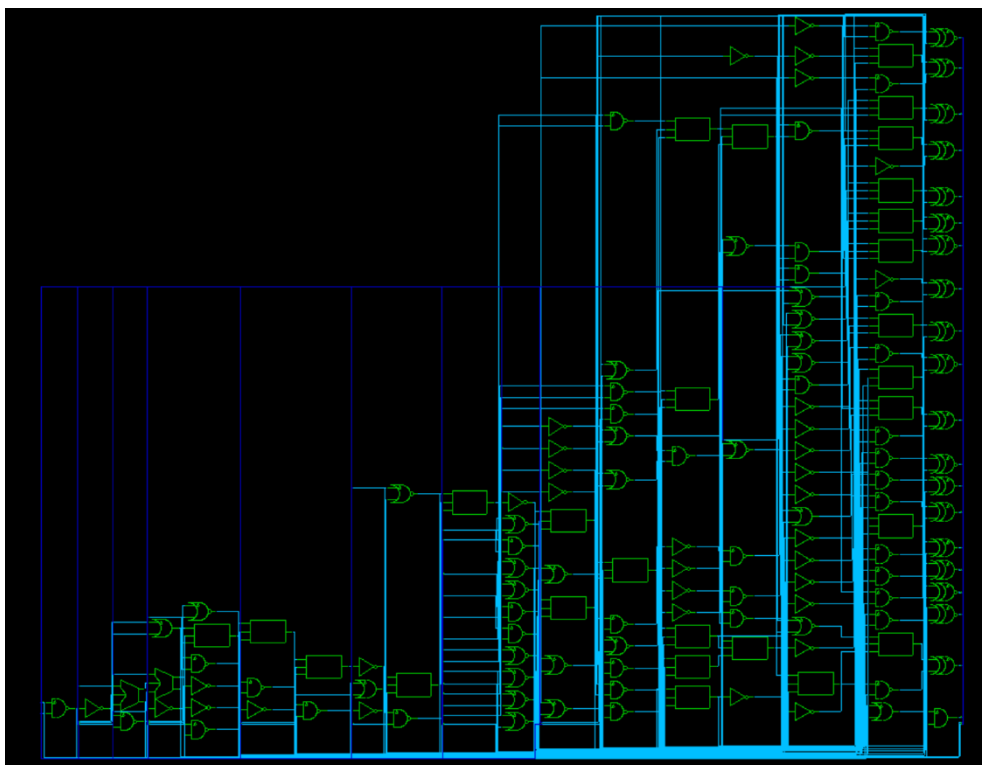
위 그림은 6.2ns로 합성했을 때의 곱셈기 내부이다. FIR 필터를 설계했을 때 합성된 곱셈

기와 비슷한 형태이다.



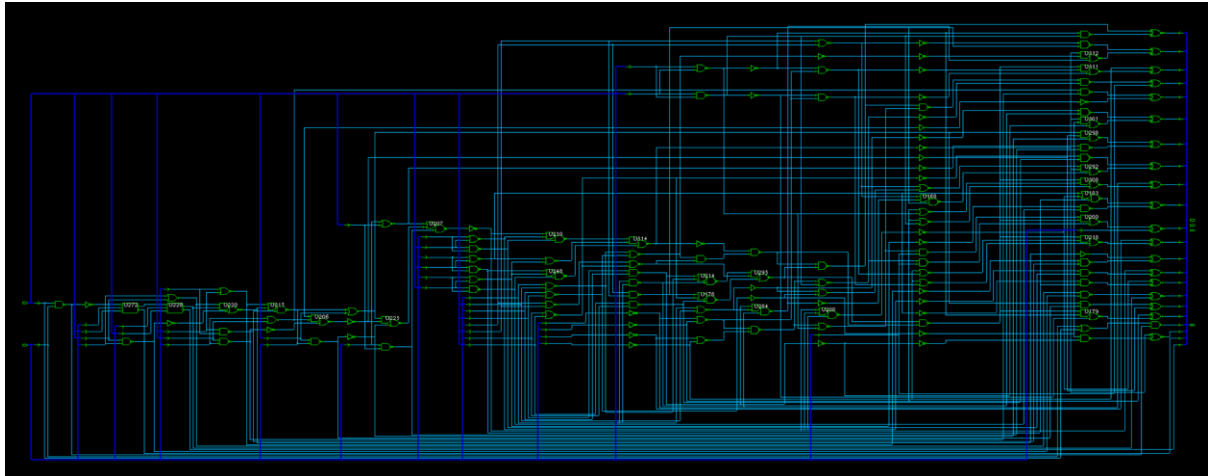
[Multiplier Schematic @ 6.1ns Period]

위 그림은 6.1ns로 합성했을 때의 곱셈기 내부이다. 역시 6.2ns 때와 완전히 다른 cell들로 합성된 것을 확인할 수 있다.



[Accumulator Schematic @ 6.2ns Period]

위 그림은 6.2ns 주기로 합성했을 때의 가산기 스키매틱이다. 같은 클럭 주기에서 곱셈기 합성에 이용된 cell들과 동일한 모양의 cell들을 여기서도 볼 수 있다.



[Accumulator Schematic @ 6.1ns Period]

이번에도 6.1ns로 합성했을 때 사용된 cell들이 다르고, 그로 인해 전체적인 형태가 다른 것을 확인할 수 있다.

4. Discussion

i. Critical Path Analysis

6.2ns로 합성했을 때의 timing report를 살펴보면 A_IN_REG -> M2/multiplier -> M2/accumulator -> M2/acc_reg가 크리티컬 패스인 것을 볼 수 있다. 스키매틱을 보고 예상했던 것과 같이 메모리 입력 레지스터부터 시작해, MAC 내부의 곱셈기와 가산기를 거쳐 누적이 이루어지는 레지스터까지의 경로가 크리티컬 패스이다. 전체 모듈을 봤을 때 가장 시간을 오래 잡아먹는 두 요소가 8비트 곱셈기와 22비트 가산기인데, 이 두 모듈이 중간에 레지스터 없이 바로 붙어있기 때문에 크리티컬 패스를 결정한다고 볼 수 있다.

한편 지금까지 만들었던 가산기나 곱셈기보다 CPD가 작은데, 이것은 게이트레벨로 설계를 하지 않고 behavioral 설계를 한 뒤 디자인 컴파일러가 최적화를 해줬기 때문이라고 볼 수 있다. 또한 사용된 곱셈기가 8비트에 불과하기 때문인 것도 영향을 미쳤을 것이다.

ii. Cause of Unexpected Area Results

더 낮은 주기 클럭으로 합성을 진행했을 때 오히려 면적이 줄어드는 현상을 발견했는데, 스키매틱 항목에서 관찰했던 것처럼 합성에 사용한 cell이 다르기 때문이라고 추측해 볼 수 있다. Cell을 다르게 사용하고, cell 사이 연결도 달라졌기 때문에 그로 인해 면적이

일반적인 경향성을 벗어났을 수도 있다. 또한 DC가 어떤 알고리즘으로 합성을 진행하는지는 알 수는 없지만, 타이밍을 맞추기 어려운 경우에 회로를 더 최적화하려고 하다가 면적이 오히려 줄었을 가능성에 대해서도 생각해볼 수 있다.